

Dicionários

Objetivo da Aula

Ao final desta aula, o aluno será capaz de:

- Entender o que são **dicionários em Python**
 - Criar e acessar dados em dicionários
 - Adicionar, alterar e remover valores
 - Percorrer dicionários com loops
 - Utilizar dicionários em cenários práticos
-

1. O que são Dicionários?

Um **dicionário** é uma estrutura de dados que armazena informações no formato **chave** → **valor**.

Estrutura:

```
chave : valor
```

Exemplo do mundo real:

```
nome : Ana  
idade : 25  
cidade : São Paulo
```

Em Python:

```
usuario= {  
    "nome": "Ana",  
    "idade": 25,  
    "cidade": "São Paulo"  
}
```

Características importantes:

- Dados são organizados por **chaves**
 - Cada chave possui um **valor associado**
 - As chaves devem ser **únicas**
-

2. Criando Dicionários

Dicionários são criados usando **chaves** `{ }`.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "ativo": True  
}  
  
print(usuario)
```

Resultado:

```
{'nome': 'Carlos', 'idade': 30, 'ativo': True}
```

3. Acessando Valores

Para acessar um valor, usamos a **chave**.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "ativo": True  
}  
  
print(usuario["nome"])  
print(usuario["idade"])
```

Saída:

Carlos
30

4. Adicionando Novos Valores

Podemos adicionar novos dados simplesmente criando uma nova chave.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30  
}  
  
usuario["cidade"] = "Curitiba"  
  
print(usuario)
```

Resultado:

```
{'nome': 'Carlos', 'idade': 30, 'cidade': 'Curitiba'}
```

5. Alterando Valores

Para alterar um valor existente, basta atribuir um novo valor à chave.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30  
}  
  
usuario["idade"] = 31  
  
print(usuario)
```

Resultado:

```
{'nome': 'Carlos', 'idade': 31}
```

6. Removendo Elementos

Usando `pop()`

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "cidade": "Curitiba"  
}  
  
usuario.pop("cidade")  
  
print(usuario)
```

Resultado:

```
{'nome': 'Carlos', 'idade': 30}
```

7. Descobrimos o Tamanho

Podemos usar `len()`.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "cidade": "Curitiba"  
}  
  
print(len(usuario))
```

Resultado:

8. Percorrendo Dicionários

Podemos percorrer as **chaves** com `for`.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "cidade": "Curitiba"  
}  
  
for chave in usuario:  
    print(chave)
```

Saída:

```
nome  
idade  
cidade
```

9. Percorrendo Chaves e Valores

Podemos usar `.items()`.

```
usuario= {  
    "nome": "Carlos",  
    "idade": 30,  
    "cidade": "Curitiba"  
}  
  
for chave, valor in usuario.items():  
    print(chave, ":", valor)
```

Saída:

```
nome : Carlos
idade : 30
cidade : Curitiba
```

10. Verificando se uma Chave Existe

Podemos usar `in`.

```
usuario= {
    "nome":"Carlos",
    "idade":30
}

if "nome" in usuario:
    print("Chave encontrada")
```

11. Aplicação Prática para QA

Imagine armazenar informações de um **teste de API**.

```
resposta_api= {
    "status_code":200,
    "mensagem":"Sucesso",
    "tempo_resposta":120
}

if resposta_api["status_code"] != 200:
    print("Erro na API")

print("Tempo de resposta:", resposta_api["tempo_resposta"])
```

Esse padrão é **extremamente comum em automação de testes**, especialmente ao validar respostas de APIs.